

**UNIVERSITY OF OKARA**

Department of Computer Science

## **DATA STRUCTURES & ALGORITHMS**

*Lab Manual*

---

### **LAB 04 | Searching Techniques: Linear Search, Binary Search, Two Pointers and Sliding Window**

---

Week 4 of 16 • Duration: 3 hours (in-lab) + home practice

*Prepared by*

**Muhammad Ramzan**

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

## About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

**Language policy:** concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

### Every lab follows the same structure

| Section                                                                                                                                                                                     | What you get                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  Objectives & CLOs                                                                                         | What you will be able to do after this lab                                                                                                                                                                                                                                                     |
|  Motivation                                                                                                | Where this topic is used in real life, industry, research and the future (2026–2030)                                                                                                                                                                                                           |
|  Concepts                                                                                                  | Simple explanations with real examples, pseudocode and dry runs                                                                                                                                                                                                                                |
|  Code                                                                                                    | Python implementation + C++ Corner                                                                                                                                                                                                                                                             |
|  Lab Tasks                                                                                               | Graded tasks:  Easy,  Medium,  Hard |
|  LeetCode                                                                                                | Real interview problems to practise                                                                                                                                                                                                                                                            |
|  Home Task & <br>Viva | Practice at home and prepare for oral questions                                                                                                                                                                                                                                                |
|  Notebooks                                                                                               | Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)                                                                                                                                                                                               |

### Lab 04 at a glance

| Item          | Details                                                                             |
|---------------|-------------------------------------------------------------------------------------|
| Lab title     | Searching Techniques: Linear Search, Binary Search, Two Pointers and Sliding Window |
| Week          | 4 of 16                                                                             |
| Prerequisites | Lab 02 (Big-O), Lab 03 (Arrays)                                                     |
| Tools         | Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner    |
| Files         | <a href="#">Lab04_Instructor.ipynb</a> , <a href="#">Lab04_Student.ipynb</a>        |
| Next lab      | Lab 05: Elementary Sorting (Bubble, Selection, Insertion)                           |



## 1. Learning Objectives

By the end of this lab, you will be able to:

1. Implement **linear search** and explain its best, average and worst case.
2. Implement **binary search** iteratively and explain why it is  **$O(\log n)$** .
3. Solve binary search **variants**: first/last occurrence, insert position, and searching on the answer.
4. Use Python's **bisect** module correctly.
5. Apply the **two-pointer** technique on sorted arrays and strings.
6. Apply the **sliding window** technique to subarray and substring problems.
7. Choose the right searching technique for a given problem.

### Course Learning Outcomes (CLO) mapping

| CLO   | Description                              | Taxonomy level |
|-------|------------------------------------------|----------------|
| CLO-2 | Implement searching algorithms           | Apply (C3)     |
| CLO-3 | Analyse and compare searching techniques | Analyse (C4)   |

## 2. Motivation: Finding Things Fast

Searching is the most frequent thing computers do. Every time you type in Google, look up a contact, check a result by roll number, or find a product on Daraz, a searching algorithm runs. The difference between a slow and a fast search is the difference between waiting minutes and getting results instantly.

### 2.1 Searching in Daily Life

| Situation                                              | Technique      | Why                                |
|--------------------------------------------------------|----------------|------------------------------------|
| Finding your name on an unsorted attendance sheet      | Linear search  | No order, so check one by one      |
| Finding a word in a dictionary book                    | Binary search  | Words are sorted alphabetically    |
| Guessing a number between 1 and 100 ("higher / lower") | Binary search  | At most 7 guesses                  |
| Finding two items whose prices add up to your budget   | Two pointers   | Prices sorted, move from both ends |
| Best 7-day period of sales in a month                  | Sliding window | Slide a fixed window over days     |

## 2.2 Searching in Industry

- **Databases** use B-tree indexes, a generalisation of binary search, to find one record among billions in a few disk reads.
- **Git** uses `git bisect` (binary search over commits) to find which commit introduced a bug.
- **Network monitoring and finance** use sliding windows to compute moving averages, traffic per minute and fraud patterns in real time.
- Two pointers and sliding window are among the **most asked interview patterns** at top companies.

## 2.3 Searching in Research

- **Machine learning:** hyperparameter search and threshold tuning often use binary search on the answer.
- **Signal and sensor processing** (e.g., Human Activity Recognition from wearable sensors) splits data into **sliding windows** of fixed length before feeding them to LSTM or CNN models.
- **Bioinformatics:** searching DNA sequences uses windows (k-mers) and sorted indexes.

## 2.4 Why It Matters in 2026–2030

### Search is everywhere in AI systems

Retrieval-Augmented Generation (RAG), the technique behind many AI assistants, first **searches** a large collection of documents and then gives the best ones to the language model. Fast, correct search is the backbone of these systems. The ideas you learn today (halving the search space, maintaining a window) appear again and again in advanced systems.

## 3. Part A: Linear Search

**Linear (sequential) search** checks every element one by one until it finds the target or reaches the end. It works on **any** array, sorted or not.

### Pseudocode

```
ALGORITHM LinearSearch(A, target)
  FOR i ← 0 TO n - 1 DO
    IF A[i] = target THEN RETURN i
  RETURN -1
```

```
def linear_search(arr, target):
    for i, value in enumerate(arr):
        if value == target:
            return i
    return -1

roll_numbers = [1043, 1017, 1088, 1002, 1056]
print(linear_search(roll_numbers, 1088), linear_search(roll_numbers, 9999))
```

| Case                  | Steps         | Big-O |
|-----------------------|---------------|-------|
| Best (first element)  | 1             | O(1)  |
| Average               | $\approx n/2$ | O(n)  |
| Worst (last / absent) | n             | O(n)  |
| Extra space           | -             | O(1)  |

## 4. Part B: Binary Search

If the array is **sorted**, we can do much better. Look at the **middle** element: if it is the target, done; if the target is smaller, it must be in the **left half**; otherwise in the **right half**. Each step **throws away half** of the remaining elements.

### **Precondition**

Binary search works **only on sorted data**. On unsorted data it gives wrong answers silently.

### Pseudocode

```
ALGORITHM BinarySearch(A, target)
    low ← 0 ; high ← n - 1
    WHILE low ≤ high DO
        mid ← (low + high) // 2
        IF A[mid] = target THEN RETURN mid
        ELSE IF A[mid] < target THEN low ← mid + 1    // go right
        ELSE high ← mid - 1                          // go left
    RETURN -1
```

**Dry run: search 23 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]**

| Step | low | high | mid | A[mid] | Decision                                                                                             |
|------|-----|------|-----|--------|------------------------------------------------------------------------------------------------------|
| 1    | 0   | 9    | 4   | 16     | $16 < 23 \rightarrow$ go right, low = 5                                                              |
| 2    | 5   | 9    | 7   | 56     | $56 > 23 \rightarrow$ go left, high = 6                                                              |
| 3    | 5   | 6    | 5   | 23     | Found at index 5  |

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

data = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(binary_search(data, 23), binary_search(data, 7))
```

**4.1 Why  $O(\log n)$ ?**

After 1 step  $n/2$  elements remain, after 2 steps  $n/4$ , after  $k$  steps  $n/2^k$ . We stop when one element remains:  $n/2^k = 1$ , so  **$k = \log_2 n$** . For 1 crore ( $10^7$ ) sorted roll numbers, binary search needs at most **24** comparisons; linear search may need 10,000,000.

```
import math
for n in [100, 10_000, 10_000_000, 8_000_000_000]:
    print(f"n = {n:>14,}: linear worst = {n:>14,}    binary worst = {math.floor(math.log2(n)) + 1}")
```

**4.2 Binary Search Variants**

Many problems need not just “is it present?” but **where** exactly. The trick is: when you find a match, **do not stop**; record it and keep searching in one direction.

```
def first_occurrence(arr, target):
    low, high, answer = 0, len(arr) - 1, -1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            answer = mid          # remember it...
            high = mid - 1        # ...but keep looking on the LEFT
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return answer

def lower_bound(arr, target):
    """First index where arr[i] >= target (= insert position)."""
    low, high = 0, len(arr)      # note: high = len(arr)
    while low < high:
        mid = (low + high) // 2
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid
    return low

grades = [50, 60, 60, 60, 75, 80, 95]
print(first_occurrence(grades, 60), lower_bound(grades, 70),
      lower_bound(grades, 100))
```

### 4.3 Python's Built-in `bisect` Module

```
import bisect
cutoffs = [50, 60, 70, 85]      # minimum marks for D, C, B, A
letters = ["F", "D", "C", "B", "A"]

def grade(marks):
    return letters[bisect.bisect_right(cutoffs, marks)]

print([grade(m) for m in [33, 50, 69, 70, 99]])

ranks = [10, 20, 30]
bisect.insort(ranks, 25)        # insert while keeping the list sorted
print(ranks)
```

### 4.4 Binary Search on the Answer

Sometimes we binary search over **possible answers** instead of an array. Example: the integer square root of  $n$  is the largest  $x$  with  $x^2 \leq n$ . Answers lie between 0 and  $n$ , and the condition “ $x^2 \leq n$ ” is True for small  $x$  and False for large  $x$ , so we can binary search it.



```
def int_sqrt(n):  
    low, high, ans = 0, n, 0  
    while low <= high:  
        mid = (low + high) // 2  
        if mid * mid <= n:  
            ans = mid  
            low = mid + 1  
        else:  
            high = mid - 1  
    return ans  
  
print(int_sqrt(10), int_sqrt(144), int_sqrt(2_000_000_000))
```

## 👉👉 5. Part C: Two Pointers

The **two-pointer** technique uses two indexes that move through the array, usually one from the **start** and one from the **end**, or both from the start at different speeds. It often turns an  $O(n^2)$  “check all pairs” solution into  **$O(n)$** .

### Example: pair with a given sum in a sorted array

You have Rs. 1,000 and a **sorted** price list. Find two items that cost exactly Rs. 1,000. If the sum is too small, move the left pointer right (bigger price). If too large, move the right pointer left.

```
def pair_with_sum(prices, budget):
    left, right = 0, len(prices) - 1
    while left < right:
        s = prices[left] + prices[right]
        if s == budget:
            return prices[left], prices[right]
        elif s < budget:
            left += 1          # need a bigger sum
        else:
            right -= 1         # need a smaller sum
    return None

print(pair_with_sum([120, 250, 300, 450, 700, 880], 1000))
```

| Step | left → price | right → price | Sum  | Action  |
|------|--------------|---------------|------|---------|
| 1    | 0 → 120      | 5 → 880       | 1000 | Found ✓ |

Try budget 950 in the notebook and trace the pointers yourself.

## 🪟 6. Part D: Sliding Window

A **sliding window** is a range `[left, right]` that moves over the array. Instead of recomputing everything for every window, we **add the element entering** and **remove the element leaving**. This makes many  $O(n \cdot k)$  problems  **$O(n)$** .

### 6.1 Fixed-Size Window

**Problem:** daily sales of a shop for 10 days are given. Find the best total over any 3 consecutive days.

**Idea**

```
Sales:    [4, 2, 12, 3, 8, 5, 1, 9, 6, 2]    k = 3
Window 1: [4, 2, 12] = 18
Slide   : remove 4, add 3  -> [2, 12, 3] = 17
Slide   : remove 2, add 8  -> [12, 3, 8] = 23   <- best
...
```

```
def best_k_days(sales, k):
    window = sum(sales[:k])
    best = window
    for right in range(k, len(sales)):
        window += sales[right] - sales[right - k]    # add new, remove old
        best = max(best, window)
    return best

print(best_k_days([4, 2, 12, 3, 8, 5, 1, 9, 6, 2], 3))
```

## 6.2 Variable-Size Window

**Problem:** length of the longest substring without repeating characters. Expand **right** one character at a time; whenever a repeat appears, shrink from **left** until the window is valid again.

```
def longest_unique(s):
    seen = set()
    left = best = 0
    for right, ch in enumerate(s):
        while ch in seen:           # shrink until ch is no longer
            repeated
            seen.remove(s[left])
            left += 1
        seen.add(ch)
        best = max(best, right - left + 1)
    return best

print(longest_unique("abcabcbb"), longest_unique("pakistan"))
```

Each character enters the window once and leaves at most once, so the total work is **O(n)** even though there is a loop inside a loop.

## 7. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int binarySearch(const vector<int>& a, int target) {
    int low = 0, high = (int)a.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;    // avoids integer overflow
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    vector<int> a = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    cout << binarySearch(a, 23) << endl;           // 5
    cout << binary_search(a.begin(), a.end(), 7) << endl;    // 0
    (false)
    cout << lower_bound(a.begin(), a.end(), 20) - a.begin() << endl;    // 5
}
```



### A famous bug

In C++ and Java,  $(low + high) / 2$  can **overflow** when low and high are very large integers. This bug existed in Java's standard library for about 9 years. Use  $low + (high - low) / 2$ . Python integers never overflow, so  $(low + high) // 2$  is safe in Python.

## 8. Which Technique When?

| Situation                                  | Technique                      | Time                    |
|--------------------------------------------|--------------------------------|-------------------------|
| Unsorted data, search once                 | Linear search                  | $O(n)$                  |
| Sorted data, search many times             | Binary search                  | $O(\log n)$             |
| Find boundary / insert position            | Binary search variant / bisect | $O(\log n)$             |
| Answer is a number with a yes/no condition | Binary search on the answer    | $O(\log(\text{range}))$ |

| Situation                                    | Technique                 | Time           |
|----------------------------------------------|---------------------------|----------------|
| Pairs in sorted data, reversing, palindromes | Two pointers              | $O(n)$         |
| Best contiguous subarray / substring         | Sliding window            | $O(n)$         |
| Many lookups of exact values                 | Set / dictionary (Lab 12) | $O(1)$ average |

## 9. Common Mistakes

| Mistake                                                           | Effect                           | Fix                               |
|-------------------------------------------------------------------|----------------------------------|-----------------------------------|
| Binary search on unsorted data                                    | Wrong answers                    | Sort first or use linear search   |
| <code>while low &lt; high</code> with <code>high = mid - 1</code> | Misses the last element          | Match loop condition with updates |
| <code>low = mid</code> instead of <code>mid + 1</code>            | Infinite loop                    | Always shrink the range           |
| Recomputing window sum every time                                 | $O(n \cdot k)$ instead of $O(n)$ | Add new element, subtract old one |
| Forgetting the empty-array case                                   | IndexError                       | Check <code>len(arr) == 0</code>  |

## 10. Lab Tasks

Solve all tasks in `Lab04_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

### Task 1 Count Comparisons

Modify linear search and binary search so that each returns `(index, comparisons)`. Search for the last element of `list(range(1, 1_000_001))` with both and print the comparison counts.

```
linear_count(list(range(1, 1001)), 1000) → (999, 1000)
```

### Task 2 First and Last Position

Given a sorted list of marks, return `[first, last]` index of a target value, or `[-1, -1]` if absent. It must be  **$O(\log n)$**  (two binary searches).

```
search_range([50, 60, 60, 60, 75], 60) → [1, 3]
```

### Task 3 Valid Palindrome (Two Pointers)

Return `True` if a sentence is a palindrome after keeping only letters and digits and ignoring case. Use two pointers that **skip** non-alphanumeric characters. Do not build a cleaned copy of the string.

```
"A man, a plan, a canal: Panama" → True
```

### Task 4 Squares of a Sorted Array

Given a sorted array that may contain negative numbers, return the squares of each number, also sorted, in  **$O(n)$** . Hint: the largest square is at one of the two ends; fill the result from the back using two pointers.

```
[-4, -1, 0, 3, 10] → [0, 1, 9, 16, 100]
```

### Task 5 Maximum Average Sensor Window

A wearable sensor records heart-rate values every second. Find the **maximum average** over any window of exactly `k` consecutive readings, in  $O(n)$ . Round to 2 decimals.

This is exactly how activity-recognition systems split sensor data into windows.

```
max_avg([1, 12, -5, -6, 50, 3], 4) → 12.75
```

### Task 6 ● Minimum Days to Make Bouquets (Search on Answer)

A nursery has flowers; `bloom[i]` is the day flower `i` blooms. You need `m` bouquets, each made of `k` **adjacent** bloomed flowers. Return the minimum day on which this is possible, or `-1`.

Hint: if it is possible on day `d`, it is also possible on any later day. Binary search the day between `min(bloom)` and `max(bloom)`, and write a helper `can_make(day)` that scans the array in  $O(n)$ .

```
min_days([1, 10, 3, 10, 2], m=3, k=1) → 3
min_days([1, 10, 3, 10, 2], m=3, k=2) → -1
```

### Task 7 ● Longest Streak with at Most One Absence

Attendance is a list of 1 (present) and 0 (absent). Find the length of the **longest continuous period** that contains **at most one** absence. Use a variable-size sliding window in  $O(n)$ .

```
[1, 1, 0, 1, 1, 1, 0, 1] → 6      # [1, 1, 0, 1, 1, 1] or [1, 1, 1, 0, 1] ...
```



## 11. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab04` folder.

| #   | Problem                                                 | Level  | Concept practised               |
|-----|---------------------------------------------------------|--------|---------------------------------|
| 704 | Binary Search                                           | Easy   | Classic binary search           |
| 35  | Search Insert Position                                  | Easy   | Lower bound                     |
| 125 | Valid Palindrome                                        | Easy   | Two pointers (Task 3)           |
| 643 | Maximum Average Subarray I                              | Easy   | Fixed window (Task 5)           |
| 34  | Find First and Last Position of Element in Sorted Array | Medium | Binary search variants (Task 2) |
| 167 | Two Sum II – Input Array Is Sorted                      | Medium | Two pointers                    |

| #    | Problem                                        | Level  | Concept practised         |
|------|------------------------------------------------|--------|---------------------------|
| 3    | Longest Substring Without Repeating Characters | Medium | Variable window           |
| 1482 | Minimum Number of Days to Make m Bouquets      | Medium | Search on answer (Task 6) |



## 12. Home Task

1. Finish all tasks and push `Lab04_Student.ipynb` to GitHub.
2. Write a **recursive** version of binary search and compare it with the iterative one (you will study recursion formally in Lab 06).
3. Implement **jump search** (jump  $\sqrt{n}$  steps, then linear search) and compare its comparison count with linear and binary search.
4. **Mini-project: Smart Result Finder.** Store 10,000 generated student records sorted by roll number. Let the user search by roll number (binary search), find all students with the same marks (first/last occurrence), and find the best 5-subject streak of a student (sliding window).
5. Solve at least 3 LeetCode problems from the list.



## 13. Viva Questions

1. What is the main requirement for binary search?
2. Why is binary search  $O(\log n)$ ? How many steps for 1,000,000 elements?
3. What are the best and worst cases of linear search?
4. When would you prefer linear search over binary search?
5. How do you find the first occurrence of a repeated value using binary search?
6. What does `bisect_left` return?
7. Explain the overflow bug in  $(low + high) / 2$  and how to avoid it.
8. What is the two-pointer technique? Give an example.
9. What is a sliding window? Why is it  $O(n)$  even with a nested while loop?
10. What is “binary search on the answer”?





## 14. Assessment Rubric

| Criteria                | Excellent (4)                               | Good (3)                  | Satisfactory (2)    | Needs work (0–1)     |
|-------------------------|---------------------------------------------|---------------------------|---------------------|----------------------|
| Correctness             | All tasks pass all tests                    | Most tasks correct        | Some tasks correct  | Mostly incorrect     |
| Logic & pseudocode      | Clear pseudocode and dry run for every task | Pseudocode for most tasks | Partial pseudocode  | None                 |
| Code quality            | Meaningful names, functions, comments       | Mostly clean              | Hard to read        | Copied or unreadable |
| Complexity & edge cases | States Big-O and handles all edge cases     | Some edge cases / Big-O   | Only basic cases    | Not considered       |
| Viva                    | Explains every line confidently             | Explains most parts       | Basic understanding | Cannot explain       |



### Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



## 15. References and Further Learning

- Python `bisect` module: [docs.python.org/3/library/bisect.html](https://docs.python.org/3/library/bisect.html)
- J. Bloch, “Extra, Extra – Read All About It: Nearly All Binary Searches and Mergesorts are Broken”, Google Research Blog, 2006.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



## About the Author

### Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

**GitHub:** [github.com/RamzanPUCIT](https://github.com/RamzanPUCIT)

**LinkedIn:** [linkedin.com/in/ramzan-ai](https://linkedin.com/in/ramzan-ai)

**ResearchGate:** [researchgate.net/profile/Muhammad-Ramzan-105](https://researchgate.net/profile/Muhammad-Ramzan-105)

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.